

LangChain: Memory

Outline

- ConversationBufferMemory
- ConversationBufferWindowMemory
- ConversationTokenBufferMemory
- ConversationSummaryMemory

ConversationBufferMemory

```
In [19]: import os

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file

# import warnings
# warnings.filterwarnings('ignore')
```

```
In [20]: from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
```

```
In [21]: llm = ChatOpenAI(temperature=0.0)
memory = ConversationBufferMemory()
conversation = ConversationChain(
    llm=llm,
    verbose=True,
    memory = memory
)
```

```
In [ ]: conversation.predict(input="Hi, my name is Andrew")
```

```
In [ ]: conversation.predict(input="What is 1+1?")
```

```
In [ ]: conversation.predict(input="What is my name?")
```

```
In [ ]: memory.load_memory_variables({})
```

```
In [ ]: memory = ConversationBufferMemory()
```

```
In [ ]: memory.save_context({"input": "Hi"},
                             {"output": "What's up"})
```

```
In [ ]: print(memory.buffer)
```

```
In [ ]: memory.load_memory_variables({})
```

```
In [ ]: memory.save_context({"input": "Not much, just hanging"},  
                             {"output": "Cool"})
```

```
In [ ]: memory.load_memory_variables({})
```

Memory

Large Language Models are 'stateless'

- Each transaction is independent

Chatbots appear to have memory by providing the full conversation as 'context'

LangChain provides several kinds of 'memory' to store and accumulate the conversation

ConversationBufferWindowMemory

```
In [23]: from langchain.memory import ConversationBufferWindowMemory
```

```
In [24]: memory = ConversationBufferWindowMemory(k=1)
```

```
In [25]: memory.save_context({"input": "Hi"},  
                              {"output": "What's up"})  
memory.save_context({"input": "Not much, just hanging"},  
                     {"output": "Cool"})
```

```
In [26]: memory.load_memory_variables({})
```

```
Out[26]: {'history': 'Human: Not much, just hanging\nAI: Cool'}
```

```
In [27]: llm = ChatOpenAI(temperature=0.0)  
memory = ConversationBufferWindowMemory(k=1) # just remember 1 conversation  
conversation = ConversationChain(  
    llm=llm,  
    memory = memory,  
    verbose=False  
)
```

```
In [ ]: conversation.predict(input="Hi, my name is Andrew")
```

```
In [ ]: conversation.predict(input="What is 1+1?")
```

```
In [ ]: conversation.predict(input="What is my name?")
```

ConversationTokenBufferMemory

```
In [30]: #!/pip install tiktoken
```

```
In [31]: from langchain.memory import ConversationTokenBufferMemory
from langchain.llms import OpenAI
llm = ChatOpenAI(temperature=0.0)
```

```
In [32]: memory = ConversationTokenBufferMemory(llm=llm, max_token_limit=30)
memory.save_context({"input": "AI is what?!"},
                    {"output": "Amazing!"})
memory.save_context({"input": "Backpropagation is what?"},
                    {"output": "Beautiful!"})
memory.save_context({"input": "Chatbots are what?"},
                    {"output": "Charming!"})
```

```
In [33]: memory.load_memory_variables({})
```

```
Out[33]: {'history': 'AI: Beautiful!\nHuman: Chatbots are what?\nAI: Charming!'}
```

ConversationSummaryMemory

```
In [34]: from langchain.memory import ConversationSummaryBufferMemory
```

```
In [ ]: # create a long string
schedule = "There is a meeting at 8am with your product team. \
You will need your powerpoint presentation prepared. \
9am-12pm have time to work on your LangChain \
project which will go quickly because Langchain is such a powerful tool. \
At Noon, lunch at the italian resturant with a customer who is driving \
from over an hour away to meet you to understand the latest in AI. \
Be sure to bring your laptop to show the latest LLM demo."

memory = ConversationSummaryBufferMemory(llm=llm, max_token_limit=100)
memory.save_context({"input": "Hello"}, {"output": "What's up"})
memory.save_context({"input": "Not much, just hanging"},
                    {"output": "Cool"})
memory.save_context({"input": "What is on the schedule today?"},
                    {"output": f"{schedule}"})
```

```
In [ ]: memory.load_memory_variables({})
```

```
In [ ]: conversation = ConversationChain(
    llm=llm,
    memory = memory,
    verbose=True
)
```

```
In [ ]: conversation.predict(input="What would be a good demo to show?")
```

```
In [ ]: memory.load_memory_variables({})
```